

CSC212 Project

Phonebook application

Team members:

1- Hussam Qannam ID: 443100831

Work: Testing and refactoring as well as writing the report.

2- Hisham Alsuhaibani ID: 443100662

Work: Lead programmer.

3- Mohammed Al Mahmud ID: 443101240

Work: Calculating the time complexity, refactoring.

Class **Contact**:

Contains attributes:

String name

String phone number

String email address

String address

String birthday

String notes

Contains methods:

Constructors.

Setters/getters

Method equalsName(String name, Boolean val)

Input: name.

Result: checks if the received name equals the name of the contact.

Output: val.

Time complexity: $O(1)$.

Method equalsPhoneNumber(String phoneNumber, Boolean val)

Input: phoneNumber.

Result: checks if the received phoneNumber equals the phoneNumber of the contact.

Output: val.

Time complexity: $O(1)$.

Method equalsEmail(String email, Boolean val)

Input: email.

Result: checks if the received email equals the email of the contact.

Output: val.

Time complexity: $O(1)$.

Method equalsAddress(String address, Boolean val)

Input: address.

Result: checks if the received address equals the address of the contact.

Output: val.

Time complexity: $O(1)$.

Method equalsBirthday(String birthday, Boolean val)

Input: birthday.

Result: checks if the received birthday equals the birthday of the contact.

Output: val.

Time complexity: $O(1)$.

Method printContact()

Input: none.

Result: prints all the contact attributes.

Output: none.

Time complexity: $O(1)$.

Class **Event**:

Contains attributes:

String title

String date

String location

String contactName

Contains methods:

Constructors.

Setters/getters.

Method **printEvent()**

Input: none.

Result: prints all event attributes.

Output: none.

Time complexity: $O(1)$.

Enum **SearchType** :

Name

Phone

Email

Address

Birthday

Notes:

This enum will later be used in a search method in the linked list which will help us in selecting a search type.

Class **Node<T>**:

Contains attributes:

Node<T> next

T data

Contains methods:

Constructor.

Class **LinkedList**:

Contains attributes:

Node<T> head

Node<T> current

Contains methods:

Constructor.

Method **findFirst()**

input: none

results: first element set as the current element.

output: none

Time complexity: $O(1)$.

Method **findNext()**

input: none

results: element following the current element is made the current element.

output: none.

Time complexity: $O(1)$.

Method **retrieve**(T e)

input: none

results: current element is copied into e.

output: T e.

Time complexity: $O(1)$.

Method **update**(T val)

input: val.

results: the element e is copied into the current node.

output: none.

Time complexity: $O(1)$.

Method **full**(boolean flag)

input: none.

results: if the number of elements in L has reached the maximum number allowed then flag is set to true otherwise false.

output: flag.

Time complexity: $O(1)$.

Method **empty**(boolean flag)

input: none.

results: if the number of elements in L is zero, then flag is set to true otherwise false.

Output: flag.

Time complexity: $O(1)$.

Method **last**(boolean flag)

input: none.

Results: if the last element is the current element, then flag is set to true otherwise false.

Output: flag.

Time complexity: $O(1)$.

Method **insert**(T val)

input: val.

results: a new node containing element e is created and inserted after the current element in the list. The new element e is made the current element. If the list is empty e is also made the head element.

output: none.

Time complexity: $O(1)$.

Method **insertSorted**(String identifier, T val)

Input: name, val

Result: it will go through the linked list to find the appropriate spot to insert the val such that the list is sorted alphabetically, when it finds the spot, it will insert using the **insert** method.

Output: none.

Time complexity: $O(n)$.

Note: we chose this method of sorting because of its time complexity, since sorting the list after inserting will result in a time complexity of $O(n^2)$.

Method **searchContact**(String val, SearchType type)

Input: String val.

Result: searches the contact list for a contact and makes it the current. using 1 of 5 SearchType (name, phone, email, address, birthday)

Output: none.

Time complexity: $O(n)$.

Method **remove()**

input: none.

results: the current element is removed from the list. If the resulting list is empty the current is set to NULL. If successor of the deleted element exists it is made the new current element, otherwise first element is made the new current element.

Output: none.

Class **Phonebook**:

Contains attributes:

LinkedList<Event> listE.

Note: we chose to store the event in a linked list to avoid writing more code, and since the linked list serves the event's goals.

LinkedList<Contact> listC.

Contains methods:

Method **isUnique**(String name, String phoneNumber, Boolean flag)

Input: name, phoneNumber.

Result: checks the contact list if there is an existing contact that holds name or phoneNumber and returns true otherwise return false.

Output: flag.

Time complexity: $O(n)$.

Method **addContact**()

Input: none.

Result: prompts the user to enter a name, phone number, email address, address, birth day, and some notes.

After that it will check if the name or the phone number already exist using the **isUnique** method. If not it will create a new contact and insert it using the **insertSortedC** method.

Output: none.

Time complexity: $O(n)$ because of **isUnique**()

Method **searchContact()**

Input: none.

Result: prompts the user to choose a search criteria,
That will be used to search for a contact in the list using the **searchContact** method in the linked list.

Output: none.

Time complexity: $O(n)$.

Method **printFirstNameContact()**

Input: none.

Result: prompts the user for a first name, and then checks the list for contacts with a matching first name.

Output: none.

Time complexity: $O(n)$.

Method **printNameAndPH(int count)**

Input: none.

Result: prints all contacts' names and phone numbers and returns the number of the contacts.

Output: count.

Time complexity: $O(n)$.

Method **deleteContact()**

Input: none.

Result: displays the contacts' names and phone numbers using **printNameAndPH** method then prompts the user for a choice.

Then the contact chosen will be removed as well as their events.

Output: none.

Time complexity: $O(n)$.

Method **isConflict**(String date, String name, String title, Boolean flag)

Input: date, name, title

Result: checks the event list for a matching date and a contact name or a matching title and a date and returns true otherwise false.

Output: flag.

Time complexity: $O(n)$.

Method **addEvent()**

Input: none.

Result: prompts the user to enter a title, contact name, date and time and a location.

After that it will check if the contact name exists using the **isUnique** method, and checks if the event conflicts with another event using the **isConflict** method, then inserts the event into the list using the insertSortedE method

Output: none.

Time complexity: $O(n)$ because of **isUnique()** and **isConflict()**

Method **searchEvent()**

Input: none.

Result: prompts the user to choose a search criteria,

That will be used to search for an event in the list, if contact name was chosen then it will display all events with that contact name.

However if event title was chosen then it will display all events with the same event title.

Output: none.

Time complexity: $O(n)$.

Method **printEvents()**

Input: none.

Result: displays all events.

Output: none.

Time complexity: $O(n)$.

Method **printSharedEventC(String contactName)**

Input: contactName.

Result: prints all event that share contactName.

Output: none.

Time complexity: $O(n)$.

Method **printSharedContactE()**

Input: none.

Result: prints all contacts that share an event title.

Output: none.

Time complexity: $O(n^2)$ since it will have to loop through two lists.

Method **printSharedEventE**(String eventTitle)

Input: eventTitle.

Result: prints all events that share an eventTitle.

Output: none.

Time complexity: $O(n)$.

Method **menu**()

input: none.

Result: prints the menu.

Output: none.

Time complexity: $O(1)$.

Class **Menu**:

This class will only have the main and its functionality is to create and use the phonebook object.